

BiC-MPPI: Bidirectional Clustering MPPI for Goal-Directed Kinodynamic Trajectory Optimization

Anonymous Authors

Abstract—Reliable goal-directed kinodynamic trajectory optimization is essential for robotic systems operating in cluttered environments. However, conventional unidirectional Model Predictive Path Integral (MPPI) methods can struggle to propagate goal-side information and preserve distinct rollout modes, reducing their reliability in challenging goal-reaching tasks. This work aims to improve goal-reaching reliability while maintaining forward dynamic consistency. We propose Bidirectional Clustering MPPI (BiC-MPPI), which retains clustered forward branches from the current state, generates reverse-time goal-side reference branches, associates representative branch pairs, and refines the resulting connections through Guide MPPI under forward dynamics. In 600 modified BARN navigation trials, BiC-MPPI achieved a 97.3% success rate, compared with 47.7–54.2% for the evaluated unidirectional baselines. In 240 simulated 6-DOF manipulator tasks, the best BiC-MPPI configuration achieved a 41.7% success rate, compared with 10.0–22.5% for the baselines. These results indicate that bidirectional clustered guidance can substantially improve the reliability of sampling-based kinodynamic trajectory optimization, while a stage-wise runtime analysis quantifies the additional cost of backward rollout generation, clustering, branch association, and Guide MPPI refinement.

Project page: [BiC-MPPI-project-page](#)

Index Terms—Motion and Path Planning, Kinematics, Optimization and Optimal Control, Model predictive path integral.

I. INTRODUCTION

GOAL-DIRECTED trajectory optimization in cluttered environments is challenging for robotic systems with nonlinear dynamics, input constraints, and collision-avoidance requirements. Sampling-based model predictive control methods, such as Model Predictive Path Integral (MPPI), are attractive because they evaluate nonlinear dynamics and nonconvex costs through independent rollouts that are well suited to GPU parallelization [4], [5]. However, standard MPPI can struggle in goal-reaching tasks: terminal costs may be weakly propagated through finite-horizon samples, and cost-weighted averaging may blend separated homotopy modes into an inferior control sequence.

Classical geometric graph- and sampling-based planners provide strong exploration mechanisms [6], [7], but their geometric solutions often require additional smoothing or tracking to satisfy differential constraints. Trajectory-optimization methods instead evaluate candidate controls directly through a system model and have been applied to real-time robotic

systems [8]–[10]. Within this class, gradient-based methods such as Differential Dynamic Programming (DDP) and its constrained variants solve local approximations through iterative forward–backward sweeps [11]–[14], whereas MPPI provides a sampling-based alternative through parallel rollout evaluation. Recent kinodynamic planners further improve long-range exploration through geometric guidance, problem-conditioned motion primitives, bidirectional informed search, and massively parallel tree expansion [15]–[19]. These methods provide effective global exploration mechanisms but generally rely on explicit search trees, graphs, learned primitive sets, or local connection procedures that are structurally distinct from rollout-based stochastic trajectory optimization. Table I summarizes representative planner classes and their practical trade-offs.

Within the MPPI framework, previous studies have improved multimodal rollout handling through clustering [20], sampling feasibility through normal–log-normal perturbations [21], and local control performance through hybridization with IPDDP or ancillary controllers [22], [23]. Recent methods have also guided forward MPPI rollouts using perception-aware objectives for alternative-path discovery [24] or separately computed object-level references for long-horizon manipulation [25]. However, these methods do not explicitly construct and associate clustered forward cost-to-come branches with reverse-time heuristic goal-side branches. This gap is nontrivial because clustering alone preserves distinct rollout modes without providing an explicit goal-side search structure, whereas directly coupling forward and backward searches introduces local-connection and two-point boundary-value difficulties [26].

This paper proposes Bidirectional Clustering MPPI (BiC-MPPI), a parallel sampling-based kinodynamic trajectory optimizer that augments forward rollout optimization with clustered reverse-time goal-side references. The backward branches are used only as heuristic structures. Branch association and Guide MPPI use these references to generate refined candidate controls that are rolled out from the current state under forward dynamics.

The contributions of this paper are threefold:

- 1) We propose a bidirectional clustering MPPI framework that retains forward cost-to-come branches from the current state and backward heuristic cost-to-go branches from the goal.
- 2) We introduce a branch-association and Guide MPPI refinement procedure that constructs reference candidates from the retained branches and refines them using rollouts under forward dynamics.

TABLE I
PLANNER-CLASS COMPARISON

Planner	Kinodynamic	Optimization	GPU parallel	Completeness / optimality	Pros	Cons
Dijkstra	×	✓	×	Complete / graph-optimal	Simple; deterministic shortest path	Graph-dependent feasibility
A*	×	✓	×	Complete / graph-optimal [†]	Efficient with good heuristic	Heuristic/resolution dependent
RRT	×	×	×	Prob. complete / not optimal	Fast exploration	Requires smoothing/tracking
Kinodynamic RRT*	✓	✓	×	Prob. complete / asympt. optimal [1]	Handles differential constraints	Requires steering/reachability assumptions
Kinodynamic State Lattice	✓	✓	×	Resolution complete / lattice-optimal [2]	Feasible motion primitives; deterministic search	Discretization/primitive-set dependent
BiC-MPPI (ours)	✓	✓	✓	No completeness / no formal optimality [‡]	Dynamics-aware; simulated cost-evaluation	Sampling-budget dependent

The GPU-parallel column indicates whether rollout-level vectorization is intrinsic to the formulation considered here. [†]With an admissible heuristic. Lattice optimality is restricted to the discretized state lattice and its motion-primitive set. [‡]Existing MPPI optimality analyses apply only under method-specific assumptions [3]; no completeness or optimality guarantee is claimed for BiC-MPPI.

3) We evaluate BiC-MPPI on modified BARN-based 2D navigation and simulated 6-DOF manipulator joint-space planning benchmarks, including stage-wise runtime decomposition and component ablations, and demonstrate 10 Hz closed-loop execution on a physical mobile robot.

The remainder of this paper is organized as follows. Section II reviews the necessary preliminaries, Section III presents BiC-MPPI, Section IV reports experiment results, and Section V concludes the paper.

II. PRELIMINARIES

A. Trajectory Optimization

For state-space trajectory optimization, we consider the state $x_t \in \mathbb{R}^n$ and the input $u_t \in \mathbb{R}^m$ with the stage cost $\psi : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}$ and the terminal state cost $\phi : \mathbb{R}^n \rightarrow \mathbb{R}$. The associated optimal control problem (OCP) for trajectory optimization can be written as

$$\min_{u_0:T-1} \phi(x_T) + \sum_{t=0}^{T-1} \psi(x_t, u_t) \quad (1a)$$

$$\text{subject to } x_0 = x_{\text{init}}, \quad (1b)$$

$$x_{t+1} = f(x_t, u_t), \quad t = 0, \dots, T-1, \quad (1c)$$

$$g(x_t) \leq 0, \quad p(x_t) \in \mathcal{F}, \quad t = 0, \dots, T, \quad (1d)$$

$$h(u_t) \leq 0, \quad t = 0, \dots, T-1 \quad (1e)$$

where x_{init} is the initial condition, f is the discrete-time dynamics, and g and h define state and input constraints. The terminal cost ϕ encodes the goal-reaching objective. The map p extracts the position component, and $\mathcal{F}$ denotes the collision-free region.

B. Model Predictive Path Integral

MPPI perturbs a nominal control sequence $\bar{U} = (\bar{u}_0, \dots, \bar{u}_{T-1})$ and evaluates the resulting rollouts from x_{init} . The k -th sampled sequence is $U^k = (u_0^k, \dots, u_{T-1}^k)$, where $u_t^k = \bar{u}_t + \varepsilon_t^k$ and $\varepsilon_t^k \sim \mathcal{N}(0, \Sigma_u)$. For horizon length H , define $\mathbb{U}_H = \{u_{0:H-1} \in \mathbb{R}^{mH} : h(u_t) \leq 0, t = 0, \dots, H-1\}$. We project sampled input sequences onto this set as

$$\mathcal{P}_{\mathbb{U}_H}(U) = \arg \min_{U' \in \mathbb{U}_H} \|U' - U\|_2. \quad (2)$$

For time-decoupled input constraints, $\mathbb{U}_H = \mathcal{U}^H$ with $\mathcal{U} = \{u \in \mathbb{R}^m : h(u) \leq 0\}$, so projection is applied element-wise. In the reported experiments, the admissible input sets are convex; otherwise, the weighted update can be reprojected onto $\mathbb{U}_H$ [27].

Let $X(U) = (x_0, \dots, x_T)$ denote the state trajectory generated by U . Collision avoidance is represented by the trajectory-level indicator

$$\mathcal{I}_{\mathcal{F}}(X) = \begin{cases} 0, & p(x_t) \in \mathcal{F} \quad \forall x_t \in X, \\ +\infty, & \text{otherwise.} \end{cases} \quad (3)$$

The MPPI objective is then

$$J(U) = \phi(x_T) + \sum_{t=0}^{T-1} \psi(x_t, u_t) + \mathcal{I}_{\mathcal{F}}(X(U)). \quad (4)$$

In the implementation, a rollout that violates a state or collision constraint is assigned the finite trajectory cost J_{col} in place of the infinite penalty. After evaluating N_s sampled trajectories, MPPI forms the update

$$J_{\min} := \min_{1 \leq \ell \leq N_s} J^\ell, \quad (5a)$$

$$\tilde{w}^k := \frac{\exp[-\gamma_u (J^k - J_{\min})]}{\sum_{\ell=1}^{N_s} \exp[-\gamma_u (J^\ell - J_{\min})]}, \quad (5b)$$

$$\hat{U} := \sum_{k=1}^{N_s} \tilde{w}^k U^k, \quad U^* := \Pi_{\mathbb{U}_T}(\hat{U}). \quad (5c)$$

where $J^k = J(U^k)$, $\gamma_u > 0$ is the inverse temperature, and $\tilde{w}^k$ is the normalized sample weight. Subtracting the minimum cost $J_{\min}$ leaves the normalized weights unchanged while improving numerical stability. The final projection enforces the input constraints. If the sampled distribution contains separated low-cost modes, a single weighted average may lie between the modes and produce an inferior or infeasible trajectory. The following subsection describes rollout clustering, which forms separate MPPI updates within the identified rollout modes.

C. Rollout Clustering MPPI

Naive MPPI returns a single weighted control sequence even when the sampled low-cost rollouts form separated modes. For example, when left- and right-turning obstacle-avoidance rollouts are averaged together, the resulting control sequence can lie between the two modes and become infeasible, although each mode contains feasible samples.

Rollout Clustering MPPI mitigates this mode-averaging problem by clustering the evaluated rollouts before applying the MPPI update [20]. For rollout r , let $X^r = X(U^r)$ denote its state trajectory and D^r denote its clustering feature. Using the indicator in (3), define the collision-free rollout-index set

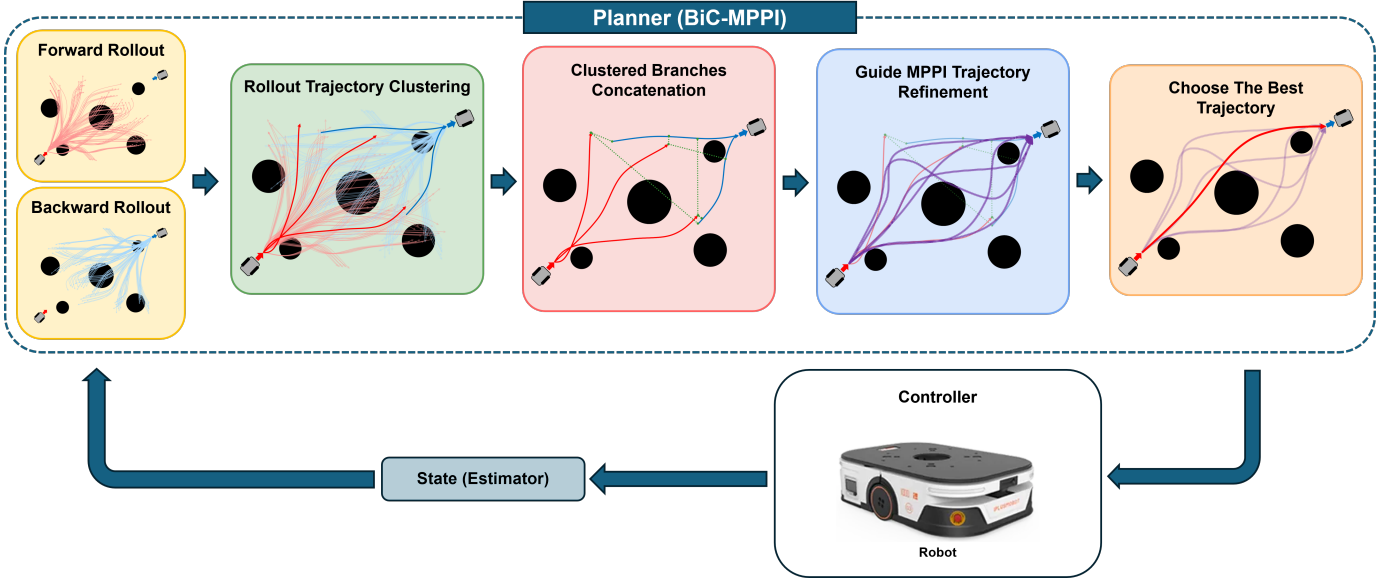


Fig. 1. Overview of the proposed BiC-MPPI framework.

as $\mathcal{V} := \{r \in \{1, \dots, N_s\} : \mathcal{I}_{\mathcal{F}}(X^r) = 0\}$. A clustering method returns a collection of disjoint rollout-index sets

$$\mathcal{C} = \{C_1, \dots, C_I\}, \quad C_i \subseteq \mathcal{V}.$$

For each cluster, the normalized weights in (5b) are renormalized within the cluster, and the representative control sequence is computed as

$$U^{C_i} = \mathcal{P}_{U_T} \left(\sum_{k \in C_i} \frac{\tilde{w}^k}{\sum_{\ell \in C_i} \tilde{w}^\ell} U^k \right). \quad (6)$$

In this work, rollout clustering can be instantiated using either K-Means [28] or DBSCAN [29]. K-Means uses a prescribed number of clusters, which provides a fixed branch count and predictable downstream computation, but requires selecting K and assigns every collision-free sample to a cluster. DBSCAN determines the cluster count from local feature density and can reject sparse samples as noise, but is sensitive to its density parameters and produces a variable number of branches.

The standard Cluster-MPPI baseline selects the lowest-cost cluster representative,

$$i^* \in \arg \min_{i \in \{1, \dots, I\}} J(U^{C_i}), \quad (C^*, U^{C^*}) := (C_{i^*}, U^{C_{i^*}}). \quad (7)$$

In BiC-MPPI, this immediate single-cluster selection is not performed in the forward and backward stages. Instead, multiple forward and backward cluster representatives are retained for the subsequent branch-association stage.

D. Bidirectional Rollout Propagation

Forward and backward rollouts are generated from the same continuous-time model

$$\dot{x} = F(x, u). \quad (8)$$

Let $\Phi_h(x, u)$ denote a one-step integration map with step size h and piecewise-constant input, implemented using explicit Euler, RK4, or another numerical integrator.

For a forward control sequence $U_f = (u_0, \dots, u_{T_f-1})$, the state is propagated from $x_0 = x_{\text{init}}$ according to

$$x_{t+1} = f_f(x_t, u_t) := \Phi_{\delta t}(x_t, u_t), \quad t = 0, \dots, T_f - 1. \quad (9)$$

For a backward control sequence $U_b = (u_0, \dots, u_{T_b-1})$, the goal-side state sequence is propagated from $x_{T_b} = x_{\text{goal}}$ toward decreasing time indices:

$$x_{t-1} = f_b(x_t, u_{t-1}) := \Phi_{-\delta t}(x_t, u_{t-1}), \quad t = T_b, \dots, 1. \quad (10)$$

The backward propagation map is not assumed to be the exact inverse of f_f . It is used only to generate heuristic goal-side branches and does not represent a physically executable reverse trajectory.

III. BiC-MPPI ALGORITHM

BiC-MPPI consists of three stages, as summarized in Fig. 1: bidirectional clustered branch generation, branch association and connection, and Guide MPPI refinement. In Step 1, forward branches are generated from the current state, while reverse-time backward branches are generated from the goal and used only as heuristic goal-side references.

Algorithm 1 gives the complete receding-horizon procedure. At each iteration, BiC-MPPI retains multiple cluster representatives in both directions, associates opposite-direction branches, refines each connected candidate using forward Guide MPPI rollouts, and selects the lowest-cost refined candidate. Throughout the algorithms, $\mathbb{Z}_{a:b} := \{a, a+1, \dots, b\}$ denotes the integer index set from a to b , where $a \leq b$.

A. Step 1: Bidirectional Clustered Branch Generation

Step 1 generates forward and backward clustered branches. Forward branches start from x_{init} and are evaluated as cost-to-come candidates, whereas backward branches start from x_{goal} and are evaluated as heuristic cost-to-go candidates. Unlike the

Algorithm 1 BiC-MPPI

```

1: Input: initial state  $x_{\text{init}}$ , goal state  $x_{\text{goal}}$ 
2: Output: executed control history  $\mathcal{U}_{\text{exec}}$ 
3: Initialize: forward seed  $U_{f0}$ ,  $\mathcal{U}_{\text{exec}} \leftarrow \emptyset$ ,  $\ell \leftarrow 0$ 
4: while goal condition is not satisfied do
5:    $U_{b0} \leftarrow \text{RandomInit}(\mathbb{U}_{T_b})$   $\triangleright$  No backward warm start
6:   Step 1. Bidirectional Clustered Branch Generation
7:    $\mathcal{B}_f = \{(U_f^i, X_f^i, J_f^i)\}_{i=1}^{I_f} \leftarrow \text{ForwardClusterMPPI}(U_{f0})$   $\triangleright$ 
   Alg. 2
8:    $\mathcal{B}_b = \{(U_b^j, X_b^j, J_b^j)\}_{j=1}^{I_b} \leftarrow \text{BackwardClusterMPPI}(U_{b0})$ 
    $\triangleright$  Alg. 2
9:   Step 2. Branch Association and Connection
10:   $\{(U_{\text{ref}}^{(i)}, X_{\text{ref}}^{(i)}, T_i)\}_{i=1}^{I_f} \leftarrow \text{Connect}(\mathcal{B}_f, \mathcal{B}_b)$   $\triangleright$  Sec. III-B
11:   $\mathcal{I}_{\text{conn}} \leftarrow \{i \in \mathbb{Z}_{1:I_f} : T_i > 0\}$ 
12:  if  $\mathcal{I}_{\text{conn}} = \emptyset$  then
13:     $i^\dagger = \arg \min_{i \in \mathbb{Z}_{1:I_f}} J_f^i$ ,  $U^{\ell,*} = U_f^{i^\dagger}$ 
14:  else
15:    Step 3. Guide MPPI Refinement
16:    for each  $i \in \mathcal{I}_{\text{conn}}$  do // Parallel
17:       $(U_g^{(i)*}, J_{\text{sel}}^{(i)}) \leftarrow \text{GuideMPPI}(U_{\text{ref}}^{(i)}, X_{\text{ref}}^{(i)})$   $\triangleright$ 
   Alg. 3
18:    end for
19:     $i^* = \arg \min_{i \in \mathcal{I}_{\text{conn}}} J_{\text{sel}}^{(i)}$ ,  $U^{\ell,*} = U_g^{(i^*)*}$ 
20:  end if
21:  Receding-horizon execution
22:   $u_0^{\ell,*} \leftarrow \text{First}(U^{\ell,*})$ 
23:  Apply  $u_0^{\ell,*}$  and observe the next state  $x_{\text{next}}$ 
24:   $\mathcal{U}_{\text{exec}} \leftarrow \mathcal{U}_{\text{exec}} \oplus u_0^{\ell,*}$ 
25:   $x_{\text{init}} \leftarrow x_{\text{next}}$ 
26:   $U_{f0} \leftarrow \text{Shift}(U^{\ell,*})$ 
27:   $\ell \leftarrow \ell + 1$ 
28: end while
29: return  $\mathcal{U}_{\text{exec}}$ 

```

clustering MPPI baseline, BiC-MPPI retains representatives from both directions for subsequent branch association.

For $q \in \{f, b\}$, let $X_q(U)$ denote the state sequence generated by the directional rollout. Using the collision indicator in (3), the forward branch-generation problem is

$$\begin{aligned}
\min_{U \in \mathbb{U}_{T_f}} \quad & J_f(U) = \phi_f(x_{T_f}) + \sum_{t=0}^{T_f-1} \psi_f(x_t, u_t) + \mathcal{I}_{\mathcal{F}}(X_f(U)) \\
\text{s.t.} \quad & x_{t+1} = f_f(x_t, u_t), \quad t = 0, \dots, T_f - 1, \\
& x_0 = x_{\text{init}}.
\end{aligned} \tag{11}$$

The backward stage retains goal-side representatives that are later associated with the forward branches. Its heuristic cost-to-go problem is

$$\begin{aligned}
\min_{U \in \mathbb{U}_{T_b}} \quad & J_b(U) = \phi_b(x_0) + \sum_{t=1}^{T_b} \psi_b(x_t, u_{t-1}) + \mathcal{I}_{\mathcal{F}}(X_b(U)) \\
\text{s.t.} \quad & x_{t-1} = f_b(x_t, u_{t-1}), \quad t = T_b, \dots, 1, \\
& x_{T_b} = x_{\text{goal}}.
\end{aligned} \tag{12}$$

Here, f_f and f_b are the positive- and negative-step numerical propagation maps defined in (9) and (10), respectively. The costs J_f and J_b score the retained initial-side and goal-side branch representatives. Algorithm 2, instantiated with $q = f$ or $q = b$, gives the corresponding directional clustering MPPI subroutine. In the numerical implementation, a rollout

Algorithm 2 Bidirectional Clustering MPPI

```

1: Input: direction  $q \in \{f, b\}$  and corresponding data
   ( $x_q, T_q, N_q, U_{q0}, J_q, \text{Rollout}_q$ )
2: Output: directional branches  $\mathcal{B}_q = \{(U_q^i, X_q^i, J_q^i)\}_{i=1}^{I_q}$ 
3: for  $k = 1, \dots, N_q$  do // Parallel
4:   Sample  $\varepsilon_{0:T_q-1}^k$  with  $\varepsilon_t^k \sim \mathcal{N}(0, \Sigma_u)$ 
5:    $U^k \leftarrow \mathcal{P}_{\mathbb{U}_{T_q}}(U_{q0} + \varepsilon^k)$ 
6:    $X^k \leftarrow \text{Rollout}_q(x_q, U^k)$ ,  $J^k \leftarrow J_q(U^k)$ 
7:   Compute the block-wise control-input clustering feature  $D^k$ 
8: end for
9:  $\mathcal{V}_q \leftarrow \{r \in \mathbb{Z}_{1:N_q} : \mathcal{I}_{\mathcal{F}}(X^r) = 0\}$ 
10: if  $\mathcal{V}_q = \emptyset$  then
11:    $\mathcal{V}_q \leftarrow \{\arg \min_{r \in \mathbb{Z}_{1:N_q}} J^r\}$ 
12: end if
13:  $\mathcal{C}_q = \{C_1, \dots, C_{I_q}\} \leftarrow \text{Clustering}(\{D^r\}_{r \in \mathcal{V}_q})$ 
14: if  $\mathcal{C}_q = \emptyset$  then
15:    $\mathcal{C}_q \leftarrow \{\mathcal{V}_q\}$ ,  $I_q \leftarrow 1$ 
16: end if
17: for each  $C_i \in \mathcal{C}_q$  do
18:    $\bar{J}_i \leftarrow \min_{r \in C_i} J^r$ ,  $w_i^s \leftarrow \exp[-\gamma_u(J^s - \bar{J}_i)]$ ,  $s \in C_i$ 
19:    $Z_i \leftarrow \sum_{s \in C_i} w_i^s$ ,  $U_q^i \leftarrow \mathcal{P}_{\mathbb{U}_{T_q}}\left(\sum_{s \in C_i} \frac{w_i^s}{Z_i} U^s\right)$ 
20:    $X_q^i \leftarrow \text{Rollout}_q(x_q, U_q^i)$ ,  $J_q^i \leftarrow J_q(U_q^i)$ 
21: end for
22: return  $\mathcal{B}_q$ 

```

violating the collision-free condition is assigned the finite total cost J_{col} in place of the infinite indicator value.

At each replanning iteration, the forward subproblem uses the shifted warm start U_{f0} , whereas the backward subproblem is reinitialized with an independently sampled control sequence U_{b0} . Given these initial sequences, the two rollout subproblems are independent and can be evaluated in parallel. They interact only in the subsequent connection stage, which pairs forward cost-to-come branches with backward cost-to-go branches.

B. Step 2: Branch Association and Connection

Step 2 associates the independently generated forward and backward branches. For each forward branch, the nearest point pair under a connection metric is found, the two branches are trimmed at the selected indices, and the resulting segments are concatenated into a guide candidate for Step 3.

The connection stage operates on the retained branch representatives output by Step 1,

$$\mathcal{B}_f = \{(U_f^i, X_f^i, J_f^i)\}_{i=1}^{I_f}, \quad \mathcal{B}_b = \{(U_b^j, X_b^j, J_b^j)\}_{j=1}^{I_b}.$$

Here, X_f^i and X_b^j are representative state trajectories after the within-cluster MPPI update. The connection search uses only the retained representatives $\mathcal{B}_f$ and $\mathcal{B}_b$; the rollout clusters and their assignments remain internal to Algorithm 2.

For each forward branch i , the associated backward branch and cut indices are

$$\begin{aligned}
(j_i, \tau_i^f, \tau_i^b) &= \arg \min_{(j, \tau, \tau')} d_c\left((X_f^i)_\tau, (X_b^j)_{\tau'}\right) \\
\text{s.t.} \quad & j \in \mathbb{Z}_{1:I_b}, \quad \tau \in \mathbb{Z}_{0:T_f}, \quad \tau' \in \mathbb{Z}_{0:T_b}.
\end{aligned} \tag{13}$$

The connection metric d_c must be defined in a consistent state space before association is performed. In the released imple-

mentation, we use a weighted Euclidean distance between the stored state vectors,

$$d_c(x, x') = \|x - x'\|_W = \sqrt{(x - x')^\top W (x - x')}, \quad (14)$$

where $W \succeq 0$ defines the state-component weighting used for branch association. Depending on the task, this metric can be instantiated as the identity-weight full-state distance, a position-only distance, or a normalized state-space distance by choosing the corresponding weights. More structured choices, such as LQR-based metrics [30], can also be substituted when a task-specific local geometry is available.

Once (13) is solved, τ_i^f and τ_i^b are the cut indices on the forward branch and selected backward branch, respectively. The two cut states need not be identical; any resulting reference discontinuity is handled by the subsequent Guide MPPI refinement.

The result of this branch association is the concatenation of the state and control input sequences:

$$\begin{aligned} U_{\text{ref}}^{(i)} &= (U_f^i)_{0:\tau_i^f-1} \oplus (U_b^{j_i})_{\tau_i^b:T_b-1}, \\ X_{\text{ref}}^{(i)} &= (X_f^i)_{0:\tau_i^f} \oplus (X_b^{j_i})_{\tau_i^b+1:T_b}. \end{aligned} \quad (15)$$

Here, $\oplus$ denotes concatenation, with $Y_{a:b} = \emptyset$ for $a > b$.

The concatenated horizon is $T_i = \tau_i^f + (T_b - \tau_i^b)$, so $|U_{\text{ref}}^{(i)}| = T_i$ and $|X_{\text{ref}}^{(i)}| = T_i + 1$. Candidates with $T_i = 0$ are excluded from $\mathcal{I}_{\text{conn}} = \{i \in \mathbb{Z}_{1:I_f} : T_i > 0\}$; if this set is empty, Algorithm 1 falls back to the lowest-cost forward branch.

C. Step 3: Guide MPPI Refinement

Step 3 refines each connected candidate with Guide MPPI. The concatenated sequence in (15) is used as a reference, and sampled forward rollouts are weighted by a guide cost.

For each connected candidate i , the Guide MPPI refinement problem is written as

$$\begin{aligned} \min_{U \in \mathcal{U}_{T_i}} \quad & J_g^{(i)}(U; X_{\text{ref}}^{(i)}) \\ \text{s.t.} \quad & x_{t+1} = f_t(x_t, u_t), \quad t = 0, 1, \dots, T_i - 1, \\ & x_0 = x_{\text{init}}. \end{aligned} \quad (16)$$

Here, $U = (u_0, \dots, u_{T_i-1})$ and $X = (x_0, \dots, x_{T_i})$ is the corresponding forward rollout. Let $J_{\text{task}}(U)$ denote the task-specific rollout objective. The Guide MPPI objective is

$$J_g^{(i)}(U; X_{\text{ref}}^{(i)}) = J_{\text{task}}(U) + w_{\text{ref}} \sum_{t=0}^{T_i} \|x_t - x_{\text{ref},t}^{(i)}\|_2. \quad (17)$$

Colliding trajectories are assigned J_{col} during both guide weighting and final candidate selection. The guide costs are used in the MPPI weight update (5), producing a refined input $U_g^{(i)*}$. Final selection removes the reference-deviation term and uses

$$J_{\text{sel}}^{(i)} = J_{\text{task,sel}}(X), \quad (18)$$

The concrete task objectives are given in the corresponding experiment subsections. Algorithm 3 summarizes the Guide MPPI refinement used after branch association.

Algorithm 3 Guide MPPI

```

1: Input: warm-start input  $U_{\text{ref}}^{(i)}$ , reference state trajectory  $X_{\text{ref}}^{(i)}$ 
2: Output: guided control input  $U_g^{(i)*}$  and selection cost  $J_{\text{sel}}^{(i)}$ 
3: for  $k = 1, \dots, N_g$  do // Parallel
4:    $\varepsilon_t^k = \varepsilon_{0:T_i-1}^k$  where  $\varepsilon_t^k \sim \mathcal{N}(0, \Sigma_u)$ ,  $t \in \mathbb{Z}_{0:T_i-1}$ 
5:    $U^k \leftarrow \mathcal{P}_{\mathcal{U}_{T_i}}(U_{\text{ref}}^{(i)} + \varepsilon^k)$ 
6:   Roll out  $X^k$  and evaluate  $J^k \leftarrow J_g^{(i)}(U^k; X_{\text{ref}}^{(i)})$ 
7: end for
8:  $\bar{J} \leftarrow \min_{k \in \mathbb{Z}_{1:N_g}} J^k$ 
9: for  $k = 1, \dots, N_g$  do
10:   $J^k = J^k - \bar{J}$ ,  $w^k = \exp(-\gamma_u J^k)$ 
11: end for
12:  $\bar{w} = \sum_{k=1}^{N_g} w^k$ ,  $\hat{U}_g^{(i)} = \sum_{k=1}^{N_g} (w^k / \bar{w}) U^k$ 
13:  $U_g^{(i)*} \leftarrow \mathcal{P}_{\mathcal{U}_{T_i}}(\hat{U}_g^{(i)})$ 
14:  $J_{\text{sel}}^{(i)} \leftarrow$  the final-selection cost in (18) after rolling out without the guide-deviation term
15: return  $(U_g^{(i)*}, J_{\text{sel}}^{(i)})$ 

```

IV. EXPERIMENTS

A. Experimental Setup and Implementation Details

All experiments were implemented in C++/CUDA and executed on an Ubuntu 22.04 workstation equipped with an AMD Ryzen 7 8845HS CPU, an NVIDIA GeForce RTX 5060 Laptop GPU, and 32 GB of system memory. In the numerical implementation, a rollout violating the collision-free condition is assigned the finite total cost $J_{\text{col}} = 10^8$ before weighting or clustering.

For both clustering variants, each sampled control sequence is represented by its block-wise mean input deviation over $B_{\text{cl}} = 3$ temporal blocks. Define

$$\begin{aligned} \mathcal{I}_r(T) &= \{t \in \mathbb{Z}_{0:T-1} : l_r(T) \leq t < u_r(T)\}, \\ l_r(T) &= \left\lfloor \frac{(r-1)T}{B_{\text{cl}}} \right\rfloor, \quad u_r(T) = \left\lfloor \frac{rT}{B_{\text{cl}}} \right\rfloor, \\ r &= 1, \dots, B_{\text{cl}}, \end{aligned} \quad (19)$$

where $\mathcal{I}_r(T)$ denotes the r -th horizon segment. The forward- and backward-stage features are

$$\begin{aligned} D_f^k &= \begin{bmatrix} \frac{1}{|\mathcal{I}_1(T_f)|} \sum_{t \in \mathcal{I}_1(T_f)} W_u(u_t^k - u_{f0,t}) \\ \frac{1}{|\mathcal{I}_2(T_f)|} \sum_{t \in \mathcal{I}_2(T_f)} W_u(u_t^k - u_{f0,t}) \\ \frac{1}{|\mathcal{I}_3(T_f)|} \sum_{t \in \mathcal{I}_3(T_f)} W_u(u_t^k - u_{f0,t}) \end{bmatrix}, \\ D_b^k &= \begin{bmatrix} \frac{1}{|\mathcal{I}_1(T_b)|} \sum_{t \in \mathcal{I}_1(T_b)} W_u(u_t^k - u_{b0,t}) \\ \frac{1}{|\mathcal{I}_2(T_b)|} \sum_{t \in \mathcal{I}_2(T_b)} W_u(u_t^k - u_{b0,t}) \\ \frac{1}{|\mathcal{I}_3(T_b)|} \sum_{t \in \mathcal{I}_3(T_b)} W_u(u_t^k - u_{b0,t}) \end{bmatrix}. \end{aligned} \quad (20)$$

Here, W_u is an input-normalization matrix and is set to the identity unless otherwise stated. The block-wise feature preserves early, middle, and late control-deviation patterns; $B_{\text{cl}} = 1$ recovers the horizon-averaged feature. Only the collision-free rollouts in the valid index set $\mathcal{V}_q$ defined in Algorithm 2 are clustered. The same valid feature set $\{D^r\}_{r \in \mathcal{V}_q}$ is provided to either K-Means or DBSCAN. For DBSCAN, two features are neighbors when $\mu_{\text{dev}} \|D^r - D^s\|_2 < \epsilon_{\text{max}}$, and a dense cluster requires at least p_{min} neighboring samples. For K-Means, the valid features are partitioned into $K_{\text{eff}} = \min(K, |\mathcal{V}_q|)$ clusters. Runtime is reported separately because BiC-MPPI adds clustering, connection search, and guide refinement beyond rollout evaluation.

TABLE II
GROUND-VEHICLE PLANNING RESULTS OVER 600 TRIALS

Method	SR [%] ↑	Coll. ↓	Iter. ↓	Total [s] ↓	Roll. [s]	Clust. [s]	Conn. [s]	Guide [s]
MPPI	54.2	18	106.0 [98.0, 113.0]	0.244 [0.226, 0.260]	0.244 [0.226, 0.260]	–	–	–
Log-MPPI	47.7	8	106.0 [97.2, 114.0]	0.202 [0.186, 0.217]	0.202 [0.186, 0.217]	–	–	–
Cluster-MPPI-DBSCAN	52.2	18	104.0 [96.0, 112.0]	1.435 [1.303, 1.619]	0.161 [0.148, 0.174]	1.273 [1.155, 1.445]	–	–
Cluster-MPPI-KMeans	47.8	13	105.0 [97.0, 113.0]	1.851 [1.678, 2.037]	<u>0.112</u> [0.105, 0.121]	<u>1.740</u> [1.573, 1.915]	–	–
BiC-MPPI-DBSCAN (ours)	<u>89.5</u>	47	<u>96.0</u> [88.0, 104.0]	2.362 [1.890, 3.413]	0.190 [0.174, 0.210]	1.966 [1.530, 3.022]	0.002 [0.001, 0.002]	0.183 [0.167, 0.202]
BiC-MPPI-KMeans (ours)	97.3	<u>10</u>	82.0 [77.0, 89.0]	3.017 [2.738, 3.264]	0.091 [0.086, 0.099]	2.342 [2.104, 2.565]	<u>0.022</u> [0.021, 0.024]	<u>0.549</u> [0.504, 0.600]

SR: success rate over all 600 trials; Coll.: collision trials; –: not applicable. Best and second-best values are shown in bold and underlined, respectively. Iteration and timing entries are reported as median [Q1, Q3] over successful trials. Stage-wise medians need not sum to the total median.

B. Ground-Vehicle Planning

The 2D navigation benchmark uses a differential-drive robot with continuous-time kinematics

$$\dot{x} = v_t \cos(\theta_t), \quad \dot{y} = v_t \sin(\theta_t), \quad \dot{\theta} = \omega_t,$$

where (x, y) is the planar position, θ is the heading, and $u_t = [v_t, \omega_t]^\top$. The OCP uses

$$0 \leq v_t \leq 1.0, \quad |\omega_t| \leq \frac{\pi}{4}, \quad (x_t, y_t) \notin \mathcal{O},$$

where $\mathcal{O}$ is the map obstacle region. The benchmark uses 300 BARN maps [31], each extended from $3\text{ m} \times 3\text{ m}$ to $3\text{ m} \times 5\text{ m}$ and inflated for footprint-aware collision checking. Two initial states, $[0.5, 0.0, \pi/2]^\top$ and $[2.5, 0.0, \pi/2]^\top$, are tested toward $x_{\text{goal}} = [1.5, 5.0, \pi/2]^\top$ on all maps, yielding 600 trials.

For a target state x_{tar} , the experiment rollout objective is

$$J(U; x_{\text{tar}}) = \sum_{t=0}^{T-1} \|x_t - x_{\text{tar}}\|_2 + \|x_T - x_{\text{tar}}\|_2. \quad (22)$$

The forward and backward stages use $x_{\text{tar}} = x_{\text{goal}}$ and $x_{\text{tar}} = x_{\text{init}}$, respectively. For a connected reference, Guide MPPI uses

$$J_g^{(i)} = \|x_{T_i} - x_{\text{goal}}\|_2 + \sum_{t=0}^{T_i} \|x_t - x_{\text{ref},t}^{(i)}\|_2. \quad (23)$$

A colliding trajectory is assigned $J_{\text{col}} = 10^8$, and final selection evaluates the goal-reaching objective without the reference-deviation term. For branch association, the SE(2) distance weights are set to $w_{xy} = 0.2$ and $w_\theta = 1.0$.

The comparison includes MPPI [4], Log-MPPI [21], and DBSCAN and K-Means variants of Cluster-MPPI [20] and BiC-MPPI, using shared Σ_u and matched branch-generation rollout-step budgets. A trial succeeds if, within 200 replanning iterations, $\|p(x_r) - p(x_{\text{goal}})\|_2 < 0.1\text{ m}$. Trials that collide are counted as failures; however, a collision-free trial also fails if it does not satisfy the goal condition within the 200-iteration limit. Elapsed computation is reported separately because BiC-MPPI performs additional stages.

Table II shows that BiC-MPPI-KM and BiC-MPPI-DB achieved success rates of 97.3% and 89.5%, respectively, compared with 47.7–54.2% for the evaluated unidirectional baselines. The runtime decomposition indicates that clustering and Guide MPPI refinement account for most of the additional computation.

TABLE III
GROUND-VEHICLE EXPERIMENT PARAMETERS

Quantity	Value
State/input dimension	$x \in \mathbb{R}^3, u \in \mathbb{R}^2$
Time step	$\Delta t = 0.1\text{ s}$
MPPI noise covariance	$\Sigma_u = \text{diag}([0.6, 0.6])$
Inverse temperature	$\gamma_u = 10$
MPPI, Log-MPPI, Cluster-MPPI	$T = 100, N = 6000$
BiC-MPPI (ours)	$T_f = 50, T_b = 50, N_f = N_b = 6000, N_g = 3000$
DBSCAN parameters	$\mu_{\text{dev}} = 1.0, \epsilon_{\text{max}} = 0.01, p_{\text{min}} = 5$
K-Means parameters	$K = 5, n_{\text{init}} = 1, n_{\text{iter}}^{\text{max}} = 100$
Map resolution	0.1 m/cell

DBSCAN and K-means use the same rollout features; only the clustering rule differs.

C. Real-Time Closed-Loop Robot Experiment

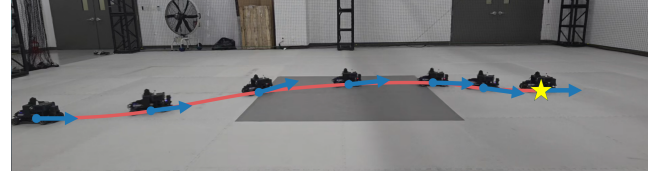


Fig. 2. Real-time closed-loop BiC-MPPI experiment. The red curve and green arrows indicate the measured robot trajectory and heading, respectively, and the yellow star denotes the goal position.

To demonstrate the real-time closed-loop applicability of BiC-MPPI, we conducted an experiment using an Ackermann-type WeBot mobile robot (WeGo Robotics) and a virtual occupancy map derived from BARN map 219. BiC-MPPI was executed on an external computer, and the first optimized control input was transmitted to the robot through ROS at each replanning iteration. The robot position and heading were measured using a motion-capture system, transformed into the virtual-map coordinate frame, and used to initialize the subsequent receding-horizon optimization. The virtual map was used for collision checking. The reported planning-latency statistics were computed over 553 replanning cycles collected from 10 closed-loop runs.

Unlike the differential-drive model used in the simulation benchmark, the physical robot was modeled using the kinematic bicycle dynamics

$$\begin{aligned} \dot{x} &= v \cos(\theta + \beta), & \dot{y} &= v \sin(\theta + \beta), \\ \dot{\theta} &= \frac{v}{l_r} \sin \beta, & \beta &= \tan^{-1} \left(\frac{l_r}{L} \tan \delta \right), \end{aligned} \quad (24)$$

where $L = 0.25\text{ m}$, $l_r = 0.13\text{ m}$, $0 \leq v \leq 1.0\text{ m/s}$, and $|\delta| \leq 0.261799\text{ rad}$. The bicycle dynamics were discretized with $\Delta t = 0.1\text{ s}$, matching the replanning period. This model was used for the forward, backward, and Guide MPPI rollouts.

The prediction horizons were set to $T_f = T_b = 30$, while the remaining cost, sampling, and DBSCAN settings were retained from the ground-vehicle benchmark. BiC-MPPI was executed with a 100 ms replanning period. The measured planning latency had a mean of 60.784 ms and ranged from 41.897 ms to 97.078 ms. The median was 52.931 ms, with first and third quartiles of 46.896 ms and 80.397 ms, respectively. Thus, all measured planning latencies remained within the prescribed 10 Hz planning deadline.

D. Manipulator Planning Benchmark

The manipulator benchmark evaluates the same solvers on simulated 6-DOF pose planning using a DH kinematic chain and a decoupled joint-space torque model. Planning is performed in configuration space, while collision checking is evaluated in task space.

The manipulator state and input are

$$x = [q^\top, \dot{q}^\top]^\top \in \mathbb{R}^{12}, \quad u = \tau \in \mathbb{R}^6,$$

where q , $\dot{q}$, and τ denote joint angles, joint velocities, and torque-like commands. Each joint follows

$$\dot{q}_i = v_i, \quad \dot{v}_i = \frac{\text{sat}(\tau_i) - d_i v_i - g_i \sin(q_i)}{I_i},$$

where $v_i = \dot{q}_i$ and $\text{sat}(\tau_i)$ clips the command by the torque limit. The coefficients are

$$\begin{aligned} I &= [4.0, 3.5, 2.5, 0.9, 0.6, 0.35], \\ d &= [3.5, 3.2, 2.5, 0.8, 0.5, 0.35], \\ g &= [0.0, 7.0, 4.5, 0.6, 0.3, 0.15]. \end{aligned} \quad (25)$$

Joint-velocity limits are $[2.0, 2.0, 2.0, 2.5, 2.5, 2.5]$ rad/s and torque limits are $[18, 18, 14, 8, 6, 4]$. Forward kinematics use DH parameters $a = [0, -0.427, -0.357, 0, 0, 0]$ m, $d_{\text{DH}} = [0.15, 0, 0, 0.11, 0.09, 0.09]$ m, and $\alpha = [\pi/2, 0, 0, \pi/2, -\pi/2, 0]$ rad. The manipulator regularization and goal-state costs are

$$\begin{aligned} \ell_{\text{reg}}(x_t, \tau_t) &= 10^{-3} \|\tau_t\|_2^2 + 2 \times 10^{-2} \|\dot{q}_t\|_2^2 \\ &\quad + 20 \ell_{\text{lim}}(q_t) + \ell_{\text{obs}}(q_t), \end{aligned} \quad (26)$$

$$\begin{aligned} \Phi(x_t, x_g) &= 1500 \|q_t - q_g\|_2^2 \\ &\quad + 500 \|p_{\text{ee}}(q_t) - p_{\text{ee}}(q_g)\|_2^2 + 100 \|\dot{q}_t\|_2^2. \end{aligned} \quad (27)$$

Here, $\ell_{\text{lim}}(q_t)$ penalizes joint configurations outside the central 85% of each admissible joint range, with a quadratic increase toward the joint limits.

The forward rollout objective is

$$J_f(U) = \sum_{t=0}^{T-1} [\ell_{\text{reg}}(x_t, \tau_t) + \Phi(x_t, x_g)] + \Phi(x_T, x_g). \quad (28)$$

The backward objective has the same form with x_{init} as the target. For a connected reference, the Guide MPPI objective is

$$\begin{aligned} J_g^{(i)} &= \sum_{t=0}^{T_i-1} [\ell_{\text{reg}}(x_t, \tau_t) + \Phi(x_t, x_g) \\ &\quad + \|x_t - x_{\text{ref},t}^{(i)}\|_2] + \Phi(x_{T_i}, x_g) \\ &\quad + \|x_{T_i} - x_{\text{ref},T_i}^{(i)}\|_2, \end{aligned} \quad (29)$$

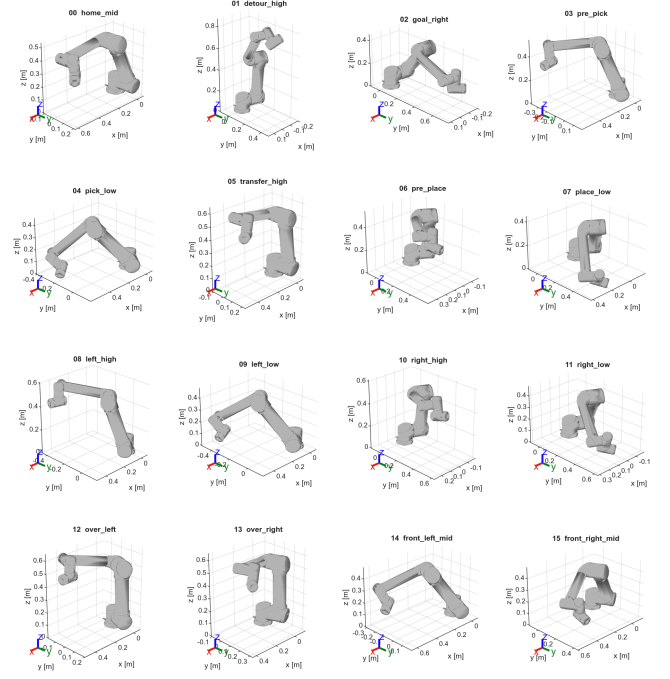


Fig. 3. Reference postures used for the manipulator pose benchmark.

and final branch selection uses

$$J_{\text{task,sel}}(X) = \sum_{t=0}^{T_i} \Phi(x_t, x_g). \quad (30)$$

Any hard collision assigns the complete trajectory cost $J_{\text{col}} = 10^8$. Each link is approximated by sampled capsule-like line segments of radius 0.045 m and checked against one AABB obstacle with 0.02 m hard-collision margin.

Tasks are generated from 16 reference postures by evaluating all ordered non-self start-goal pairs, giving $16 \times 15 = 240$ tasks. The postures are converted to zero-velocity states, and one AABB obstacle is placed between each start-goal pair. The obstacle is defined in the world frame as $\mathcal{O}_{\text{box}} = \{p \in \mathbb{R}^3 \mid c_{\text{obs}} - h_{\text{obs}} \leq p \leq c_{\text{obs}} + h_{\text{obs}}\}$, where c_{obs} is the midpoint of the start and goal end-effector positions and h_{obs} is the obstacle half-size. For all manipulator trials, the obstacle half-size was set to $h_{\text{obs}} = [0.1, 0.1, 0.1]^\top$ m. All solvers share the same task instance and warm-start policy. Fig. 3 shows the reference postures.

A trial is counted as a strict success if

$$\begin{aligned} \|q - q_g\| &< 0.03 \text{ rad}, \\ \|p_{\text{ee}}(q) - p_{\text{ee}}(q_g)\| &< 0.05 \text{ m}, \\ \|\dot{q}\| &< 0.2 \text{ rad/s}, \end{aligned}$$

and the executed trajectory remains collision-free.

TABLE IV
MANIPULATOR PLANNING RESULTS

Method	SR [%]	Joint [%]	EE [%]	CF [%]	Iter. [steps]	Time [s]
MPPI	20.8	25.0	37.9	70.8	135.0	<u>3.821</u>
Log-MPPI	22.1	25.4	37.5	72.1	134.5	3.805
Cluster-MPPI-KM	10.0	11.3	19.2	72.1	141.8	5.817
Cluster-MPPI-DB	22.5	25.8	38.3	72.1	134.8	5.617
BiC-MPPI-KM (ours)	41.7	47.1	54.6	73.8	92.1	11.599
BiC-MPPI-DB (ours)	40.0	46.7	52.9	73.3	85.5	4.554

KM: K-Means; DB: DBSCAN; CF: collision-free rate. Success uses strict joint, end-effector, velocity, and collision-free conditions. Iteration and wall time are averaged over successful trials only.

TABLE V
MANIPULATOR PLANNING PARAMETERS

Quantity	Value
State/input dimension	$x \in \mathbb{R}^{12}, u \in \mathbb{R}^6$
Time step	$\Delta t = 0.02$ s
MPPI noise covariance	$\Sigma_u = \text{diag}([4.5, 4.5, 3.8, 2.2, 1.8, 1.2])$
Inverse temperature	$\gamma_u = 0.0015$
MPPI, Log-MPPI, Cluster-MPPI	$T = 90, N = 1024$
BiC-MPPI (ours)	$T_f = 45, T_b = 45, N_f = N_b = N_g = 1024$
DBSCAN parameters	$\mu_{\text{dev}} = 1.0, \epsilon_{\text{max}} = 3.8, p_{\text{min}} = 8$
K-Means parameters	$K = 5, n_{\text{init}} = 1, n_{\text{iter}}^{\text{max}} = 100$

Table IV shows that BiC-MPPI-KM and BiC-MPPI-DB achieved success rates of 41.7% and 40.0%, respectively, compared with 10.0–22.5% for the evaluated unidirectional baselines. The higher success rates of BiC-MPPI are accompanied by additional computation: BiC-MPPI-DB requires 4.554 s per successful trial, compared with 3.805–5.817 s for the evaluated forward-only methods.

E. Ablation and Analysis

Ablation is evaluated on the same modified BARN-based 2D navigation benchmark and success criterion used in the main ground-vehicle experiment. Table VI compares two reference baselines with component variants of BiC-MPPI. MPPI and Cluster-MPPI-DB are included as contextual baselines rather than one-at-a-time ablations. Full BiC-MPPI uses K-Means, which achieved the highest success rate in Table II.

The BiC-MPPI w/o Guide variant retains bidirectional branch generation and association but directly selects the connected candidate using the trajectory cost without Guide MPPI refinement. The BiC-MPPI w/o Cluster variant replaces clustering with the eight lowest-cost rollouts in each direction, which are treated as individual branch representatives.

TABLE VI
COMPONENT ABLATION

Method	SR [%] $\uparrow$	Failures $\downarrow$	Iter. [steps]	Time [s]
MPPI	54.2	275	106.2	0.244
Cluster-MPPI-DB	52.2	287	104.7	1.495
BiC-MPPI w/o Cluster	92.7	44	98.0	0.415
BiC-MPPI-KM w/o Guide	88.2	71	84.0	3.544
Full BiC-MPPI-KM	97.3	16	83.8	3.017

Iterations and times are averaged over successful trials. Cluster-MPPI uses DBSCAN and Full BiC-MPPI uses K-means.

Full BiC-MPPI achieved the highest reliability among the evaluated configurations. Removing Guide MPPI reduced the

success rate by 9.1 percentage points, indicating that branch association alone does not provide the same reliability as forward refinement around the connected references. Replacing clustering with the eight lowest-cost directional rollouts reduced the success rate by 4.6 percentage points, while retaining substantially higher reliability than the unidirectional baselines. These results are consistent with retained bidirectional candidates providing useful goal-side guidance, with clustering and Guide MPPI refinement yielding additional improvements.

V. CONCLUSIONS AND FUTURE WORK

This paper presented BiC-MPPI, a bidirectional clustering MPPI framework that introduces heuristic goal-side structure into goal-directed kinodynamic trajectory optimization while recovering forward dynamic consistency through Guide MPPI refinement. In 600 modified BARN navigation trials, the best BiC-MPPI configuration achieved a 97.3% success rate, compared with 47.7–54.2% for the evaluated unidirectional baselines. In 240 simulated 6-DOF manipulator tasks, the best BiC-MPPI configuration achieved a 41.7% success rate, compared with 10.0–22.5% for the baselines. The runtime decomposition and component ablations indicate that retained bidirectional branch diversity and Guide MPPI refinement both contribute to the observed reliability improvement.

Future work will focus on reducing the computational overhead through adaptive sample and branch allocation, extending the method to dynamic environments, and validating fully onboard execution with physical obstacles and onboard perception. We will also investigate dynamics-aware connection metrics to better characterize the conditions under which bidirectional guidance provides the largest benefit.

REFERENCES

- [1] D. J. Webb and J. Van Den Berg, “Kinodynamic RRT*: Asymptotically optimal motion planning for robots with linear dynamics,” in *2013 IEEE International Conference on Robotics and Automation*, 2013, pp. 5054–5061.
- [2] M. Pivtoraiko and A. Kelly, “Differentially constrained motion replanning using state lattices with graduated fidelity,” in *2008 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2008, pp. 2611–2616.
- [3] H. Homburger, F. Messerer, M. Diehl, and J. Reuter, “Optimality and suboptimality of MPPI control in stochastic and deterministic settings,” *IEEE Control Systems Letters*, vol. 9, pp. 763–768, 2025.
- [4] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, “Information-theoretic model predictive control: Theory and applications to autonomous driving,” *IEEE Transactions on Robotics*, vol. 34, no. 6, pp. 1603–1622, 2018.
- [5] M. Kazim, J. Hong, M.-G. Kim, and K.-K. K. Kim, “Recent advances in path integral control for trajectory optimization: An overview in theoretical and algorithmic perspectives,” *Annual Reviews in Control*, vol. 57, p. 100931, 2024.
- [6] H. Choset, K. M. Lynch, S. Hutchinson, G. A. Kantor, and W. Burgard, *Principles of robot motion: theory, algorithms, and implementations*. MIT Press, 2005.
- [7] S. M. LaValle, *Planning algorithms*. Cambridge University Press, 2006.
- [8] R. Chai, A. Savvaris, A. Tsourdos, and S. Chai, “Overview of trajectory optimization techniques,” in *Design of Trajectory Optimization Approach for Space Maneuver Vehicle Skip Entry Problems*. Springer, 2019, pp. 7–25.
- [9] A. Gasparetto, P. Boscariol, A. Lanzutti, and R. Vidoni, “Path planning and trajectory planning algorithms: A general overview,” *Motion and operation planning of robotic systems: Background and practical approaches*, pp. 3–27, 2015.

- [10] H.-H. Kwon, H.-S. Shin, Y.-H. Kim, and D.-H. Lee, "Trajectory optimization for impact angle control based on sequential convex programming," *The Transactions of The Korean Institute of Electrical Engineers*, vol. 68, no. 1, pp. 159–166, 2019.
- [11] Y. Tassa, N. Mansard, and E. Todorov, "Control-limited differential dynamic programming," in *2014 IEEE International Conference on Robotics and Automation*, 2014, pp. 1168–1175.
- [12] Z. Xie, C. K. Liu, and K. Hauser, "Differential dynamic programming with nonlinear constraints," in *2017 IEEE International Conference on Robotics and Automation*, 2017, pp. 695–702.
- [13] A. Pavlov, I. Shames, and C. Manzie, "Interior point differential dynamic programming," *IEEE Transactions on Control Systems Technology*, vol. 29, no. 6, pp. 2720–2727, 2021.
- [14] T. A. Howell, B. E. Jackson, and Z. Manchester, "Altro: A fast solver for constrained trajectory optimization," in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2019, pp. 7674–7679.
- [15] J. Chatrola, A. Ajith, K. Leahy, and C. Chamzas, "Multi-layer motion planning with kinodynamic and spatio-temporal constraints," in *Proceedings of the 28th ACM International Conference on Hybrid Systems: Computation and Control*, 2025, pp. 1–7.
- [16] J. Franke, A. Moldagalieva, P. Hanfeld, and W. Hönig, "Accelerating db-A* for kinodynamic motion planning using diffusion," *arXiv preprint arXiv:2503.05539*, 2025.
- [17] Y. Wang, B. Mu, S. Shokouhi, and M.-W. Thein, "Optimal kinodynamic motion planning through anytime bidirectional heuristic search with tight termination condition," *arXiv preprint arXiv:2604.11587*, 2026.
- [18] N. Perrault, Q. H. Ho, and M. Lahijanian, "Kino-PAX⁺: Near-optimal massively parallel kinodynamic sampling-based motion planner," *arXiv preprint arXiv:2602.02846*, 2026.
- [19] Y. Gao, A. Lu, and Z. Kingston, "Fast asymptotically optimal kinodynamic planning via vectorization," *arXiv preprint arXiv:2607.03987*, 2026.
- [20] S. Patrick and E. Bakolas, "Path integral control with rollout clustering and dynamic obstacles," in *2024 American Control Conference*, 2024, pp. 809–814.
- [21] I. S. Mohamed, K. Yin, and L. Liu, "Autonomous navigation of agvs in unknown cluttered environments: log-mppi control strategy," *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 10 240–10 247, 2022.
- [22] M.-G. Kim, M. Jung, J. Hong, and K.-K. K. Kim, "MPPI-IPDDP: A hybrid method of collision-free smooth trajectory generation for autonomous robots," *IEEE Transactions on Industrial Informatics*, vol. 21, no. 7, pp. 5037–5046, 2025.
- [23] E. Trevisan and J. Alonso-Mora, "Biased-MPPI: Informing sampling-based model predictive control by fusing ancillary controllers," *IEEE Robotics and Automation Letters*, vol. 9, no. 6, pp. 5871–5878, 2024.
- [24] Y. Zhai, R. Reiter, and D. Scaramuzza, "PA-MPPI: Perception-aware model predictive path integral control for quadrotor navigation in unknown environments," *IEEE Robotics and Automation Letters*, 2026.
- [25] N. Raicevic, B. R. Radhakrishnan, C. Yu, K. M. B. Lee, and N. Atanasov, "Object-informed model predictive path integral control for non-prehensile robot manipulation," *arXiv preprint arXiv:2605.30778*, 2026.
- [26] S. Nayak and M. W. Otte, "Bidirectional sampling-based motion planning without two-point boundary value solution," *IEEE Transactions on Robotics*, vol. 38, no. 6, pp. 3636–3654, 2022.
- [27] H. H. Bauschke and J. M. Borwein, "On projection algorithms for solving convex feasibility problems," *SIAM review*, vol. 38, no. 3, pp. 367–426, 1996.
- [28] S. Lloyd, "Least squares quantization in PCM," *IEEE transactions on information theory*, vol. 28, no. 2, pp. 129–137, 1982.
- [29] M. Ester, H.-P. Kriegel, J. Sander, X. Xu *et al.*, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *kdd*, vol. 96, no. 34, 1996, pp. 226–231.
- [30] R. Tedrake, "Lqr-trees: Feedback motion planning on sparse randomized trees," 2009.
- [31] D. Perille, A. Truong, X. Xiao, and P. Stone, "Benchmarking metric ground navigation," in *2020 IEEE International Symposium on Safety, Security, and Rescue Robotics*, 2020, pp. 116–121.